S.No: 1	Exp. Name: ***Even or Odd***	Date: 2026-09-08

Aim:

Write a program to check whether the given number is even or odd.

Input Format:

- A single line containing an integer n, representing the number to be checked.

Output Format:

- If n is even, print "Even".
- Otherwise, print "Odd".

Note:

- This question does not have partial weightages for test cases. All the test cases must be passed.

Source Code:

q102990/evenOrOdd.java

```
package q102990;
import java.util.Scanner;

public class evenOrOdd {
        // Write your code here
        public static void main(String[] args){
                        Scanner obj = new Scanner(System.in);
                        int num = obj.nextInt();
                        if(num % 2 == 0){
                                System.out.println("Even");
                        }else{
                                System.out.println("Odd");
                        }
                }
}
```

Execution Results - All test cases have succeeded!

ID: 25ME1A5483

Ramachandra College of Engineering

Test Case - 1
User Output
14
Even

Test Case - 2
User Output
-7
Odd

S.No: 2	Exp. Name: ***Sum of digits***	Date: 2026-09-08

Aim:

Write a Java program that prompts the user to enter a positive integer and calculates the sum of its digits using a loop.

Input Format:

- The input is a positive integer.

Output Format:

- The output is an integer that represents the sum of the digits of the given integer.

Source Code:

q22633/SumOfDigits.java

```
package q22633;

import java.util.Scanner;

public class SumOfDigits {
        // write your code here..
        public static void main(String args[]){
                Scanner obj = new Scanner(System.in);
                int num = obj.nextInt();
                int sum = 0;
                while(num > 0){
                        sum = sum + num % 10;
                        num = num / 10;
                }
                System.out.println(sum);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
345
12

Test Case - 2
User Output
9879
33

S.No: 3	Exp. Name: ***Default Values of Primitive Datatypes using Static Keyword***	**Date: 2026-09-08**

Aim:

Write a Java program that demonstrates the default values of different data types.

When instance variables or static variables are declared without explicit initialization, Java automatically assigns default values.

Your task is to:

- Declare static variables of the following primitive data types (**byte**, **short**, **int**, **long**, **float**, **double**, **char**, **boolean**) and one reference type (**String**) without assigning any values.
- Print the default values of these variables.

Input Format:

- There is no input required for this program.

Output Format:

- The program should display the default values of all primitive data types and the **String** reference in the following format:

```
byte: <value>
short: <value>
int: <value>
long: <value>
float: <value>
double: <value>
char: [<value>]
boolean: <value>
String: <value>
```

Note: Print all static default values by adding their corresponding variable names after each concatenation operator (+).

Source Code:

ID: 25ME1A5483

Ramachandra College of Engineering

Q69851/DefaultValues.java

```
package Q69851;
public class DefaultValues {
        // Write your code here...
        static byte defaultByte;
        static short defaultShort;
        static int defaultInt;
        static long defaultLong;
        static float defaultFloat;
        static double defaultDouble;
        static char defaultChar;
        static boolean defaultBoolean;
        static String defaultString;

        public static void main(String args[]){
                System.out.println("byte: "+defaultByte  );
                System.out.println("short: "+defaultShort  );
                System.out.println("int: "+defaultInt  );
                System.out.println("long: "+defaultLong  );
                System.out.println("float: "+defaultFloat  );
                System.out.println("double: "+defaultDouble  );
                System.out.println("char: ["+defaultChar  +"]");
                System.out.println("boolean: "+defaultBoolean  );
                System.out.println("String: "+defaultString  );
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
byte: 0
short: 0
int: 0
long: 0
float: 0.0
double: 0.0
char: []
boolean: false
String: null

S.No: 4	Exp. Name: ***Sizes of Data Types***	Date: 2026-09-08

Aim:

Write a Java program that prints the sizes (in bytes) of all primitive data types. The program should also print "Hello World" as its first output.

Input Format:

- There is no input required for this program.

Output Format:

- The program should produce the following output on the first line:

```
Hello World
```

- Subsequent lines: The size (in `bytes`) of each primitive data type in Java, formatted as:

```
Size of byte: <size> bytes
Size of short:<size> bytes
Size of int: <size> bytes
Size of long: <size> bytes
Size of float: <size> bytes
Size of double: <size> bytes
Size of char: <size> bytes
```

Source Code:

sizeOfDataTypes.java

```
public class sizeOfDataTypes {
    // write your code here
        public static void main(String args[]){

                System.out.println("Hello World");

                System.out.println("Size of byte: "+ Byte.BYTES+"
bytes");
                System.out.println("Size of short: "+
Short.BYTES+" bytes");
                System.out.println("Size of int: "+
Integer.BYTES+" bytes");
                System.out.println("Size of long: "+ Long.BYTES+"
bytes");
                System.out.println("Size of float: "+
Float.BYTES+" bytes");
                System.out.println("Size of double: "+
Double.BYTES+" bytes");
                System.out.println("Size of char: "+
Character.BYTES+" bytes");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Hello World
Size of byte: 1 bytes
Size of short: 2 bytes
Size of int: 4 bytes
Size of long: 8 bytes
Size of float: 4 bytes
Size of double: 8 bytes
Size of char: 2 bytes

S.No: 5	Exp. Name: ***Remove Duplicates and Sort Array***	Date: 2026-09-08

Aim:

Write a Java program to remove all duplicate elements from an array. Sort the remaining unique elements in ascending order and print them.

Input Format:

- The first line contains an integer N, representing the size of the array.
- The second line contains N space-separated integers representing the elements of the array.

Output Format:

- A single line containing the unique, space-separated elements sorted in ascending order.

Note: Do not print any trailing space after the last element. The output should end with a newline.

Source Code:

q110110/RemoveDuplicates.java

```
package q110110;
import java.util.Scanner;
import java.util.Arrays;

public class RemoveDuplicates {
        // Write your code here...
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);
                int n = sc.nextInt();
                int[] arr = new int[n];
                for(int i = 0;i < n;i++){
                        arr[i] = sc.nextInt();
                }
                Arrays.sort(arr);
                for(int i = 0;i < n;i++){
                        if(i == 0 || arr[i] != arr[i - 1]){
                                if(i != 0)
                                        System.out.print(" ");
                                System.out.print(arr[i]);
                        }
                }
                System.out.println();
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
45 12 89 12 33 45
12 33 45 89

Test Case - 2
User Output
5
10 20 30 40 50

10 20 30 40 50

S.No: 6	Exp. Name: ***Implementing String Buffer***	Date: 2026-09-08

Aim:

Write a Java program that uses a **StringBuffer** to delete a character from a given string based on the position provided by the user. The program should:

- Accept a string as input from the user.
- Accept an integer position to delete a character at that position.
- Validate the position. If the position is valid, remove the character and print the updated string.

Input Format:

- The first line contains a string `inputString` (may contain spaces).
- The second line contains an integer representing the position of the character to be deleted (0-based index).

Output Format:

- If the position is valid, print the updated string after removing the character.
- If the position is invalid, print "invalid position"

Source Code:

q55527/StringBufferExample.java

```
package q55527;
import java.util.Scanner;

public class StringBufferExample {
        // Write your code here...
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);
                String inputString = scanner.nextLine();
                int position = scanner.nextInt();
                StringBuffer stringBuffer = new
StringBuffer(inputString);
                if(position >= 0 && position <
stringBuffer.length()){
                        stringBuffer.deleteCharAt(position);
                        System.out.println(stringBuffer);
                }
                else{
                        System.out.println("invalid position");
                }
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Java Programming
15
Java Programmin

Test Case - 2
User Output
abcd 1234
20
invalid position

Test Case - 3
User Output
Programming Language
11
ProgrammingLanguage

S.No: 7	Exp. Name: ***String Operations***	Date: 2026-09-08

Aim:

Write a Java program to perform basic string operations such as finding the length of a string, concatenating two strings, and comparing two strings.

The program should:

- Read two strings from the user.
- Display the length of the first string.
- Concatenate the two strings.
- Compare the two strings using the `equals()` method and display whether they are equal.

Input Format:

- The first line contains a string representing the first string.
- The second line contains a string representing the second string.

Output Format:

Display the results in the following format:

```
Length of First String: <length>
Concatenated String: <concatenated_string>
Strings are Equal
```

or

```
Length of First String: <length>
Concatenated String: <concatenated_string>
Strings are Not Equal
```

where:

- <length> is the number of characters in the first string.
- <concatenated_string> is the result of joining the first and second strings without any additional spaces.

Constraints:

- Both strings may contain alphabets, digits, and special characters.
- Comparison is case-sensitive.

Source Code:

q109442/StringOperations.java

```
package q109442;
import java.util.Scanner;

public class StringOperations {
        // write your code here
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);

                String str1 = sc.nextLine();
                String str2 = sc.nextLine();

                System.out.println("Length of First String:
"+str1.length());
                System.out.println("Concatenated String:
"+str1.concat(str2));

                if(str1.equals(str2)){
                        System.out.println("Strings are Equal");
                }
                else{
                        System.out.println("Strings are Not
Equal");
                }
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Java
Java
Length of First String: 4
Concatenated String: JavaJava
Strings are Equal

Test Case - 2

User Output
Java
java
Length of First String: 4
Concatenated String: Javajava
Strings are Not Equal

S.No: 8	Exp. Name: ***Java Program to Define a Student Class and Display Student Details***	**Date: 2026-09-08**

Aim:

Write a Java program to define a **Student** class with the following specifications:

Attributes (Private):

- **name**: Name of the student(String).
- **rollNo**: Roll number of the student(int).
- **branch**: Branch of study of the student(String).
- **marks**: Marks obtained by the student(int).

Constructors:

- **Student()**: Default constructor that initializes name, rollNo, branch, and marks to default values.
- **Student(String name)**: Parameterized constructor with one parameter (name), initializing name and setting other attributes to default values.
- **Student(String name, int rollNo)**: Parameterized constructor with two parameters (name and rollNo), initializing name, rollNo, and setting branch and marks to default values.
- **Student(String name, int rollNo, String branch, int marks)**: Parameterized constructor initializes all attributes based on provided values.

Methods:

- void **showStudentDetails():** Displays the details of the student, including **name, roll number, branch, and marks**.
- int **getMarks():** Returns the **marks** of the student

Input Format:

The program should prompt the user to enter the following details:

```
Name of the Student (a string)
Roll Number (an integer)
Branch of Study (a string)
Marks Obtained (an integer)
```

Output Format:

Output the student details exactly as shown below:

- Display the **name, roll number, branch, and marks** of the student in separate lines formatted as specified.

Source Code:

q35212/Student.java

```
package q35212;
import java.util.Scanner;
class Student {
    // Write your code here...
        private String name;
        private int rollNo;
        private String branch;
        private int marks;

        public Student(){
                name="";
                rollNo=0;
                branch="";
                marks=0;
        }

        public Student(String name) {
                this.name=name;
                rollNo=0;
                branch="";
                marks=0;
        }

        public Student(String name, int rollNo){
                this.name=name;
                this.rollNo=rollNo;;
                branch="";
                marks=0;
        }

        public Student(String name, int rollNo,String branch, int
marks){
                this.name=name;
                this.rollNo=rollNo;;
                this.branch=branch;
                this.marks=marks;
        }

        public void showStudentDetails(){
                System.out.println("Name: "+name);
                System.out.println("Roll Number: " + rollNo);
                System.out.println("Branch: " + branch);
                System.out.println("Marks: " + marks);
        }
```

```
        public int getMarks(){
                return marks;
        }
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);

                System.out.print("Name: ");
                String name = scanner.nextLine();

                System.out.print("Roll Number: ");
                int rollNo = scanner.nextInt();
                scanner.nextLine();

                System.out.print("Branch: ");
                String branch = scanner.nextLine();

                System.out.print("Marks: ");
                int marks = scanner.nextInt();

                Student student = new Student(name, rollNo,
branch, marks);
                System.out.println("Student Details:");
                student.showStudentDetails();

                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Name:
John
Roll Number:
10
Branch:
Computer Science
Marks:
85
Student Details:

Name: John
Roll Number: 10
Branch: Computer Science
Marks: 85

S.No: 9	Exp. Name: ***Method Overloading***	Date: 2026-09-19

Aim:

Write a Java program to calculate the area of different geometric shapes using method overloading. The program should be able to calculate:

- The area of a circle is calculated using the radius.
- The area of a rectangle is calculated using the length and width.
- The area of a square is calculated using the side length.

The program should prompt the user to input the relevant dimensions and then print the area of the **shape**.

Input Format:

- A double value for the radius of the **circle**.
- A double value for the length of the **rectangle**.
- A double value for the width of the **rectangle**.
- An integer value for the side length of the **square**.

Output Format:

- The area of the **circle** is formatted to two decimal places.
- The area of the **rectangle** is formatted to two decimal places.
- The area of the **square** is formatted to two decimal places.

Sample Input:

```
radius of the circle: 5.0
length of the rectangle: 4.0
width of the rectangle: 6.0
side length of the square: 3
```

Sample Output:

```
Area of the circle: 78.54
Area of the rectangle: 24.00
Area of the square: 9.00
```

Note:

- Use **Math.PI** in Java to represent the constant **π (pi)** when calculating the area of a circle, instead of using an approximate value like 3.14.
- Refer to the visible test cases to strictly match the input/output format.

Source Code:

q62248/MethodOverloadingExample.java

```
package q62248;
import java.util.Scanner;

class Shape {

        // Method to calculate the area of a circle (using
radius)
        double calculateArea(double radius){
                return Math.PI * radius * radius;
        }

        // Method to calculate the area of a rectangle (using
length and width)
        double calculateArea(double length,double width){
                return length * width;
        }

        // Method to calculate the area of a square (using  side
length)
        double calculateArea(int side){
                return side * side;
        }
}
public class MethodOverloadingExample{
        public static void main(String[] args){
                Shape shape = new Shape();
                Scanner scanner = new Scanner(System.in);
                System.out.print("radius of the circle: ");
                double radius = scanner.nextDouble();
                double circleArea = shape.calculateArea(radius);
                System.out.printf("Area of the circle: %.2f\n",
circleArea);
                System.out.print("length of the rectangle: ");
                double length = scanner.nextDouble();
                System.out.print("width of the rectangle: ");
                double width = scanner.nextDouble();
                double rectangleArea =
shape.calculateArea(length,width);
                System.out.printf("Area of the rectangle:
%.2f\n",rectangleArea);
                System.out.printf("side length of the square: ");
                int side = scanner.nextInt();
                double squareArea = shape.calculateArea(side);
                System.out.printf("Area of the square:
```

```
%.2f\n",squareArea);
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
radius of the circle:
3.5
Area of the circle: 38.48
length of the rectangle:
0
width of the rectangle:
5
Area of the rectangle: 0.00
side length of the square:
0
Area of the square: 0.00

Test Case - 2
User Output
radius of the circle:
10.0
Area of the circle: 314.16
length of the rectangle:
5.3
width of the rectangle:
2.3
Area of the rectangle: 12.19
side length of the square:
10
Area of the square: 100.00

S.No: 10	Exp. Name: ***Implementation of Constructor***	**Date: 2026-09-19**

Aim:

Write a Java program that demonstrates the use of constructors in a class. The program should perform the following tasks:

Create a class **Person** with the following attributes:

- name (String)
- age (int)

Add a constructor to the **Person** class that initializes the **name** and **age** attributes using the values passed as parameters.
Create a method **displayDetails()** in the Person class to print the details of the person.

Input Format:

- The first line contains a string representing the name of the person.
- The second line contains an integer representing the age of the person.

Output Format:

Print the name and age of the person in the following format:

```
Name: <name>
Age: <age>
```

Source Code:

q55542/Constructor.java

```
package q55542;
import java.util.Scanner;

class Person {
    //write your code here
        String name;
        int age;

        Person(String name,int age) {
                this.name=name;
                this.age=age;
        }
        void displayDetails(){
                System.out.println("Name: "+name);
                System.out.println("Age: "+age);
        }
}

public class Constructor{
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);
                String name = scanner.nextLine();
                int age = scanner.nextInt();
                Person person = new Person(name,age);
                person.displayDetails();
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Alice
25
Name: Alice
Age: 25

Test Case - 2

User Output
Bob
29
Name: Bob
Age: 29

S.No: 11	Exp. Name: ***Constructor Overloading and Default Constructor***	**Date: 2026-09-19**

Aim:

You are given a Java program that demonstrates the concept of constructor overloading.

The class **Test** contains:

1. Two private integer variables i and j.
2. Three overloaded constructors:
   - A **default constructor** that sets both i and j to 0.
   - A **constructor with one parameter** that sets i to the given value and j to 0.
   - A **constructor with two parameters** that sets both i and j using the given values.
   - Implement a method **display()** that prints the values of i and j.
3. The program creates three objects of the **Test** class
   - **obj1** - created using the default constructor.
   - **obj2** - created using the constructor with one input value for i.
   - **obj3** - created using the constructor with two input values for i and j.

Input Format:

- First, **obj1** uses the default constructor - no input.
- After displaying **obj1**, read an integer from the user - this is used for i in **obj2**.
- After displaying **obj2**, read two integers - these are used for i and j in **obj3**.

Output Format:

- The program displays the values of i and j for each object in the following format:

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:<i_value>
Displaying values for obj2:
Value of i: <value>
Value of j: 0
Enter a value for i:<i_value>
Enter a value for j:<j_value>
Displaying values for obj3:
Value of i: <i_value>
Value of j: <j_value>
```

Sample Test Case (Combined Input and Output):

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i: 10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i: 40
Enter a value for j: 50
Displaying values for obj3:
Value of i: 40
Value of j: 50
```

Note:

- Input and output are combined in the test cases. Please ensure your program takes input and displays output simultaneously, in the same sequence as shown in the sample interaction.
- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q21947/Main.java

```
package q21947;
import java.util.Scanner;
class Test {
    // Write your code here...
        // private instance variables
        private int i;
        private int j;
        //default constructor
        Test(){
                i=0;
                j=0;
        }
        //parameterized constructor with one parameter
        Test(int i){
                this.i=i;
                this.j=0;
        }
        //parameterized constructor with two parameters
        Test(int i,int j){
                this.i=i;
                this.j=j;
        }
        //display function to print the values of i and j
        void display(){
                System.out.println("Value of i: "+i);
                System.out.println("Value of j: "+j);
        }
}
public class Main{
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);
                Test obj1 = new Test();
                System.out.println("Displaying values for
obj1:");
                obj1.display();
                System.out.print("Enter a value for i: ");
                int value1 = scanner.nextInt();
                Test obj2 = new Test(value1);
                System.out.println("Displaying values for
obj2:");
                obj2.display();
                System.out.print("Enter a value for i: ");
                int value2 = scanner.nextInt();
                System.out.print("Enter a value for j: ");
```

```
                int value3 = scanner.nextInt();
                Test obj3 = new Test(value2,value3);
                System.out.println("Displaying values for
obj3:");
                obj3.display();
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i:
40
Enter a value for j:
50
Displaying values for obj3:
Value of i: 40
Value of j: 50

Test Case - 2
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0

```
Enter a value for i:
20
Enter a value for j:
30
Displaying values for obj3:
Value of i: 20
Value of j: 30
```

S.No: 12	Exp. Name: ***Person's name and age - Single Inheritance***	Date: 2026-09-19

Aim:

Write a Java program to demonstrate inheritance by creating two related classes: **Person** and **Citizen**.

Class **Person:**

Create a class named **Person** with the following characteristics:

- A String variable **name**.
- A method **inputName()** that takes user input for the name.
- A method **displayName()** that prints the name to the console.

Class **Citizen**:

Create a class named **Citizen** that inherits from **Person** and has the following characteristics:

- An **int** variable **age**.
- A method **inputAge()** that takes user input for the age.
- A method **displayAge()** that prints the age to the console.

Input/Output Format:

- First, prompt the user with: "Enter name: " followed by a string representing the name.
- Then, display the name preceded by "Name: ".
- On the next line, prompt: "Enter age: " followed by an integer representing the age.
- Finally, display the age preceded by "Age: ".

Sample Input/Output:

```
Enter name: John doe
Name: John doe
Enter age: 23
Age: 23
```

Source Code:

q23042/MainPerson.java

```
package q23042;
import java.util.Scanner;

        // write your code here..

class Person {
        String name;
        int age;
        void inputName(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Enter name: ");
                name = sc.nextLine();
        }
        void displayName(){
                System.out.println("Name: "+name);
        }
}
class Citizen extends Person{
        void inputAge(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Enter age: ");
                age = sc.nextInt();
        }
        void displayAge(){
                System.out.println("Age: "+age);
        }
}
public class MainPerson{
        public static void main(String[] args){
        Citizen citizen = new Citizen();
        citizen.inputName();
        citizen.displayName();
        citizen.inputAge();
        citizen.displayAge();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Enter name:
John doe
Name: John doe
Enter age:
23
Age: 23

Test Case - 2
User Output
Enter name:
Shreya
Name: Shreya
Enter age:
38
Age: 38

S.No: 13	Exp. Name: ***Multilevel Inheritance in Java***	**Date: 2026-09-22**

Aim:

Write a Java program to show multilevel inheritance to manage student information and calculate their marks. The program should include the following functionalities:

Student Class:

This class should contain two attributes:

- **id (int)**: Represents the student's ID.
- **name (String):** Represents the student's name.

Methods:

- **setData(int id, String name):** To set the student's ID and name.
- **displayData():** To display the student's ID and name.

Marks Class (inherits from Student):

This class should contain the following marks:

- **javaMarks (float):** Marks obtained in Java.
- **cMarks (float):** Marks obtained in C.
- **cppMarks (float):** Marks obtained in C++.

Methods:

- **setMarks(float javaMarks, float cMarks, float cppMarks) :** To set the marks for Java, C, and C++.
- **displayMarks():** To display the marks for Java, C, and C++.

Result Class (inherits from Marks):

This class should calculate the total and average of the marks.

Methods:

- **compute()** : To calculate the total and average marks.
- **showResult()**: To display the total and average marks.

Input Format:

The program should prompt the user for the following inputs:

- Student ID (integer)
- Student Name (string)
- Java Marks (float)
- C Marks (float)
- C++ Marks (float)

Output Format:

The program should output the following:

- Student ID in the format "Id : <id>"
- Student Name in the format "Name : <name>"
- Java Marks in the format "Java marks : <java marks>"
- C Marks in the format "C marks : <c marks>"
- C++ Marks in the format "Cpp marks : <cpp marks>"
- Total Marks in the format "Total : <total>"
- Average Marks in the format "Avg : <avg>"

Source Code:

MultilevelInheritance.java

```
import java.util.Scanner;

// Write your code here

class Student{
        int id;
        String name;
        void setData(int id,String name){
                this.id=id;
                this.name=name;
        }
        void displayData(){
                System.out.println("Id : "+id);
                System.out.println("Name : "+name);
        }
}

class Marks  extends Student{
        float javaMarks;
        float cMarks;
        float cppMarks;
        void setMarks(float javaMarks,float cMarks,float
cppMarks){
                this.javaMarks = javaMarks;
                this.cMarks = cMarks;
                this.cppMarks = cppMarks;
        }
        void displayMarks(){
                System.out.println("Java marks : "+javaMarks);
                System.out.println("C marks : "+cMarks);
                System.out.println("Cpp marks : "+cppMarks);
        }
}

class Result extends Marks{
        float total;
        float average;
        void compute(){
                total = javaMarks + cMarks + cppMarks;
                average = total/3;
        }
        void showResult(){
                System.out.println("Total : "+total);
                System.out.println("Avg : "+average);
```

```
        }
}

public class MultilevelInheritance{
        // Write your code here...
        public static void main(String args[]) {
                Scanner s = new Scanner(System.in);
                Scanner sc = new Scanner(System.in);
                Result r = new Result();
                System.out.print("Id: ");
                int id = s.nextInt();
                System.out.print("Name: ");
                String name = sc.nextLine();
                r.setData(id, name);
                System.out.print("Java marks: ");
                float jv = s.nextFloat();
                System.out.print("C marks: ");
                float c = s.nextFloat();
                System.out.print("CPP marks: ");
                float cpp = s.nextFloat();
                r.displayData();
                r.setMarks(jv,c,cpp);
                r.displayMarks();
                r.compute();
                r.showResult();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Id:
1001
Name:
Yamuna
Java marks:
75.5
C marks:
96.5
CPP marks:

Ramachandra College of Engineering | 2025-2029-AIDS-B | ID: 25ME1A5483 | Page No: 39

87
Id : 1001
Name : Yamuna
Java marks : 75.5
C marks : 96.5
Cpp marks : 87.0
Total : 259.0
Avg : 86.333336

Test Case - 2
User Output
Id:
1000
Name:
Ganga
Java marks:
77
C marks:
85.5
CPP marks:
96.5
Id : 1000
Name : Ganga
Java marks : 77.0
C marks : 85.5
Cpp marks : 96.5
Total : 259.0
Avg : 86.333336

S.No: 14	Exp. Name: ***Area of Shapes using Abstract Classes***	Date: 2026-09-22

Aim:

Write a Java program that defines an abstract class **Shape** with three subclasses (**Rectangle**, **Triangle**, **Circle**). Each subclass calculates and prints the area of the respective shape based on user-provided dimensions. Implement the program as described below:

Define an abstract class **Shape** with the following attributes and methods:

Attributes:

- **dimension1** (double)
- **dimension2** (double)

Constructor:

- **Shape(double dimension1, double dimension2)** – initializes **dimension1** and **dimension2**.

Methods:

- **public abstract void printArea()** - an abstract method to calculate and print the area of the shape.

Implement three subclasses of **Shape**:

- **Rectangle:** Computes area using the formula $length \times width$. Invoke the superclass constructor using **super(length, width)**.
- **Triangle:** Computes area using the formula $0.5 \times base \times height$. Invoke the superclass constructor using **super(base, height)**.
- **Circle:** Computes area using the formula $Math.PI \times radius \times radius$. Since a circle requires only one dimension, use **dimension1** to store the radius and initialize **dimension2** to 0 by invoking **super(radius, 0)**.

Input and Output Format:

1.**Rectangle**

- The program first prompts the user with:

```
Length and Width of Rectangle:
```

- The user enters two double values representing the length and width of the rectangle.
- Immediately after reading the input, the program prints:

```
Area of Rectangle: <area formatted to two decimal places>
```

2.**Triangle**

- Next, the program prompts the user with:

```
Base and Height of Triangle:
```

- The user enters two double values representing the base and height of the triangle.
- Immediately after reading the input, the program prints:

```
Area of Triangle: <area formatted to two decimal places>
```

3.**Circle**

- Finally, the program prompts the user with:

`Radius of Circle:`

- The user enters one double value representing the radius of the circle.
- Immediately after reading the input, the program prints:

`Area of Circle: <area formatted to two decimal places>`

Note: Refer to the sample test cases for print statements to be entered

Source Code:

q35629/Main.java

```
package q35629;
import java.util.Scanner;

// Write your code here...
abstract class Shape{
        double dimension1, dimension2;
        Shape(double dimension1,double dimension2){
                this.dimension1 = dimension1;
                this.dimension2 = dimension2;
        }
        public abstract void printArea();
}

class Rectangle extends Shape{
        Rectangle(double length,double width){
                super(length,width);
        }
        public void printArea(){
                double area = dimension1 * dimension2;
                System.out.printf("Area of Rectangle:
%.2f\n",area);
        }
}

class Triangle extends Shape{
        Triangle(double base,double height){
                super(base,height);
        }
        public void printArea(){
                double area = 0.5 * dimension1 * dimension2;
                System.out.printf("Area of Triangle:
%.2f\n",area);
        }
}

class Circle extends Shape{
        Circle(double radius){
                super(radius,0);
        }
        public void printArea(){
                double area = Math.PI * dimension1 * dimension1;
                System.out.printf("Area of Circle: %.2f\n",area);
        }
}
```

Ramachandra College of Engineering | 2025-2029-AIDS-B | ID: 25ME1A5483 | Page No: 43

```
public class Main {
    // Write your code here...
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);

                System.out.print("Length and Width of Rectangle:
");
                double length = scanner.nextDouble();
                double width = scanner.nextDouble();
                Rectangle rectangle = new Rectangle(length,
width);
                rectangle.printArea();

                System.out.print("Base and Height of Triangle:
");
                double base = scanner.nextDouble();
                double height = scanner.nextDouble();
                Triangle triangle = new Triangle(base, height);
                triangle.printArea();

                System.out.print("Radius of Circle: ");
                double radius = scanner.nextDouble();
                Circle circle = new Circle(radius);
                circle.printArea();

                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Length and Width of Rectangle:
5.5 3.5
Area of Rectangle: 19.25
Base and Height of Triangle:
10 20
Area of Triangle: 100.00
Radius of Circle:
6.25

Area of Circle: 122.72

Test Case - 2
User Output
Length and Width of Rectangle:
10 20
Area of Rectangle: 200.00
Base and Height of Triangle:
20 30
Area of Triangle: 300.00
Radius of Circle:
40
Area of Circle: 5026.55

S.No: 15	Exp. Name: ***Super Keyword***	Date: 2026-09-22

Aim:

Write a Java program that demonstrates the use of inheritance and the super keyword to perform addition and subtraction of two integers.

In this program, you are required to implement a parent–child class relationship where a child class reuses the functionality of its parent class using the super keyword.

Class Requirements

1. Parent Class: **Parents**
   - Create a class named **Parents**.
   - Implement a method **add(int num1, int num2)** that accepts two integers as parameters.
   - Prints their sum in the format:

```
Addition: <sum>
```

2. Child Class: **Child**
   - Create a class named **Child** that extends the **Parents** class.
   - Implement a method **sub(int num1, int num2)** that calls the **add()** method of the parent class using the `super` keyword.
   - Calculates and prints the subtraction of the two integers in the format:

```
Subtraction: <difference>
```

Input Format:

- A single line containing two integers separated by a space.

Output Format:

- The first line prints the addition of the two numbers.
- The second line prints the subtraction of the two numbers.

Sample Input:

```
2 3
```

Sample Output:

```
Addition: 5
Subtraction: -1
```

Source Code:

q18020/SumandSub.java

```
package q18020;
// Write your code here...
import java.util.Scanner;
class Parents{
        void add(int num1,int num2){
                int sum = num1 + num2;
                System.out.println("Addition: "+sum);
        }
}

class Child extends Parents{
        void sub(int num1,int num2){
                super.add(num1,num2);
                int difference = num1 - num2;
                System.out.println("Subtraction: "+difference);
        }
}
public class SumandSub {

    // Write your code here...
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);
                int num1 = sc.nextInt();
                int num2 = sc.nextInt();
                Child obj = new Child();
                obj.sub(num1, num2);

                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2 3
Addition: 5
Subtraction: -1

Ramachandra College of Engineering | 2025-2029-AIDS-B | ID: 25ME1A5483 | Page No: 47

Test Case - 2
User Output
15 96
Addition: 111
Subtraction: -81

S.No: 16	Exp. Name: ***Implementation of Multiple Inheritance using Interface***	**Date: 2026-09-22**

Aim:

You are required to implement the **Circle** class, which demonstrates multiple inheritance using interfaces. The provided **Shape** and **Color** interfaces define methods that need to be implemented within the **Circle** class.

1. **Shape** Interface:

The **Shape** Interface contains two methods:

- **double calculateArea():** To calculate the area of the circle.
- **double calculatePerimeter():** To calculate the perimeter of the circle.

The **Color** Interface contains one method:

- **String getColor():** To get the color of the circle.

Circle **Class**

- Create a class named **Circle** that implements both the **Shape** and **Color** interfaces.
- The class should contain:
  - A double variable to store the radius of the circleA String variable to store the color of the circle
- The class must provide method bodies for all methods declared in the interfaces.
- Implement the calculations using the following formulas:
- $Area = \pi \times radius \times radius Perimeter = 2 \times \pi \times radius$

Input Format:

The program prompts the user for input in the following order:

```
Radius of the circle:   //reads a double value
Color of the circle:   //reads a String value
```

Output Format:

- After reading the input, the program prints:

```
Circle Area: <area formatted to two decimal places>
Circle Perimeter: <perimeter formatted to two decimal places>
Circle Color: <color>
```

Note:

- Use **Math.PI** for π value.

Source Code:

q27463/MultipleInheritance.java

```
package q27463;
import java.util.Scanner;
//write your code here...
interface Shape{
        double calculateArea();
        double calculatePerimeter();
}
interface Color{
        String getColor();
}
class Circle implements Shape, Color{
        private double radius;
        private String color;
        Circle(double radius,String color){
                this.radius = radius;
                this.color = color;
        }
        public double calculateArea(){
                return Math.PI * radius * radius;
        }
        public double calculatePerimeter(){
                return 2 * Math.PI * radius;
        }
        public String getColor(){
                return color;
        }
}
public class MultipleInheritance {
        //write your code here...
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);
                System.out.print("Radius of the circle: ");
                double radius = scanner.nextDouble();
                scanner.nextLine();
                System.out.print("Color of the circle: ");
                String color = scanner.nextLine();
                Circle circle = new Circle(radius, color);
                System.out.printf("Circle Area:
%.2f%n",circle.calculateArea());
                System.out.printf("Circle Perimeter:
%.2f%n",circle.calculatePerimeter());
                System.out.println("Circle Color: " +
circle.getColor());
```

```
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Radius of the circle:
2.0
Color of the circle:
red
Circle Area: 12.57
Circle Perimeter: 12.57
Circle Color: red

Test Case - 2
User Output
Radius of the circle:
5.0
Color of the circle:
violet
Circle Area: 78.54
Circle Perimeter: 31.42
Circle Color: violet

Test Case - 3
User Output
Radius of the circle:
3.0
Color of the circle:
blue
Circle Area: 28.27
Circle Perimeter: 18.85
Circle Color: blue

S.No: 17	Exp. Name: ***Runtime Polymorphism***	Date: 2026-09-22

Aim:

Write a Java program that demonstrates runtime polymorphism using inheritance and method overriding.

Problem Description:

- You are required to create an **Animal** class with a method **makeSound()**. Then, create two subclasses, **Dog** and **Cat**, which override the **makeSound()** method to provide their own implementation.
- The program should prompt the user to input the type of animal, either "Dog" or "Cat". Based on the input, the program should create an object of the corresponding subclass (either Dog or Cat), and call the **makeSound()** method to print the sound made by that animal.

Classes to be created:

Animal Class:

- Method **makeSound()**: Prints "Animal makes a sound".

Dog Class (extends Animal):

- Override the **makeSound()** method to print "Dog barks".

Cat Class (extends Animal):

- Override the **makeSound()** method to print "Cat meows".

Input Format:

- The input will consist of a single string that can either be "Dog" or "Cat".

Output Format:

- If the input is "Dog", print:

```
Dog barks
```

- If the input is "Cat", print:

```
Cat meows
```

- If the input is neither "Dog" nor "Cat", print:

```
invalid animal
```

Source Code:

q56422/RuntimePolymorphism.java

```
package q56422;
import java.util.Scanner;

// Write your code here...

class Animal{
        void makeSound(){
                System.out.println("Animal makes a sound");
        }
}

class Dog extends Animal{
        void makeSound(){
                System.out.println("Dog barks");
        }
}

class Cat extends Animal{
        void makeSound(){
                System.out.println("Cat meows");
        }
}

public class RuntimePolymorphism {
        // Write your code here...
        public static void main(String[] args){
                Scanner scanner = new Scanner(System.in);
                String animalType = scanner.nextLine();
                Animal animal = null;
                if(animalType.equalsIgnoreCase("Dog")){
                        animal = new Dog();
                }else if(animalType.equalsIgnoreCase("Cat")){
                        animal = new Cat();
                }else{
                        System.out.println("invalid animal");
                        scanner.close();
                        return;
                }
                animal.makeSound();
                scanner.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Dog
Dog barks

Test Case - 2
User Output
Goat
invalid animal

S.No: 18	Exp. Name: ***Array accessing***	Date: 2026-10-02

Aim:

Write a Java program that handles array index access with proper error handling.

Input Format:

- The first line is the integer that represents the size of the array.
- The second line is the integers separated by space.
- The third line is the integer that represents the index at which the user wants to access the array element.

Output Format:

- If the index is valid, output a single line in the format: "Element at index <index> is: <element>".
- If the index is invalid, output a single line: "Error: Index out of bounds".

Constraint:

- $0 \leq index < size$

Note:

- Refer to sample test cases for better understanding.

Source Code:

q28186/ArrayIndexHandler.java

```
package q28186;
import java.util.Scanner;

public class ArrayIndexHandler {
        // write the code...
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                int size = sc.nextInt();
                int[] arr = new int[size];
                for(int i = 0;i < size;i += 1){
                        arr[i] = sc.nextInt();
                }
                int index = sc.nextInt();
                if(index >= 0 && index < size){
                        System.out.printf("Element at index %d
is: %d\n", index, arr[index]);
                }else{
                        System.out.println("Error: Index out of
bounds");
                }
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
78956
2
Element at index 2 is: 9

S.No: 19	Exp. Name: ***Exception Handling***	Date: 2026-10-04

Aim:

Write a Java program that demonstrates the use of Exception Handling for the following scenarios:

1. Divide by zero (`ArithmeticException`)
2. Accessing an invalid index in an array (`ArrayIndexOutOfBoundsException`)
3. Using a method on a null object (`NullPointerException`)

The program should prompt the user for inputs in sequence, perform each operation, and handle exceptions using try-catch blocks, printing appropriate error messages or results.

Input and Output Format (in order of execution):

Division Operation:

Input:

- First line prompts "Enter the numerator: " followed by an integer input for the numerator.
- The second line prompts "Enter the denominator: " followed by an integer input for the denominator.

Output:

- If the denominator is 0, print: "ArithmeticException: Division by zero is not allowed."
- Otherwise, print: "Result of division: <result>"

Note: Handle `ArithmeticException` for division by zero.

Array Access Operation:

Input:

- Third line prompts "Enter the index to access in the array: " followed by an integer input.
- Use a predefined array {1, 2, 3, 4, 5}.

Output:

- If the index is invalid, print: "ArrayIndexOutOfBoundsException: Index is out of bounds."
- Otherwise, print: "Value at index <index>: <value>"

Note: Handle `ArrayIndexOutOfBoundsException` for invalid indices.

String Length Operation:

Input:

- Fourth line prompts "Enter a string (or 'null' to simulate null pointer exception): " followed by a string input
- If the input is the literal string "null", treat it as a null reference.

Output:

- If the string is null: "NullPointerException: Cannot perform operations on a null reference."
- Otherwise, print: "Length of the string: <length>"

Note:

- Handle `NullPointerException` when attempting to call `.length()` on a null string.
- Refer to the visible test-cases for better understanding.

Source Code:

ExceptionDemo.java

```
import java.util.Scanner;
class ExceptionDemo {
    // write your code here...
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);

                System.out.print("Enter the numerator: ");
                int numerator = sc.nextInt();

                System.out.print("Enter the denominator: ");
                int denominator = sc.nextInt();

                try{
                        int result = numerator / denominator;
                        System.out.println("Result of division:
"+result);
                }catch(ArithmeticException e){
                System.out.println("ArithmeticException: Division
by zero is not allowed.");
                }
                int[] array = {1,2,3,4,5};
                System.out.print("Enter the index to access in
the array: ");
                int index = sc.nextInt();
                try{
                        System.out.println("Value at index " +
index + ": " + array[index]);
                }catch(ArrayIndexOutOfBoundsException e){

System.out.println("ArrayIndexOutOfBoundsException: Index is out
of bounds.");
                }
                sc.nextLine();
                System.out.print("Enter a string (or 'null' to
simulate null pointer exception): ");
                String input = sc.nextLine();

                String str;

                if(input.trim().equals("null")){
                        str = null;
                }else{
                        str = input;
                }
```

```
                try{
                        System.out.println("Length of the string:
"+str.length());
                }catch(NullPointerException e){
                        System.out.println("NullPointerException:
Cannot perform operations on a null reference.");
                }

                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the numerator:
6
Enter the denominator:
3
Result of division: 2
Enter the index to access in the array:
1
Value at index 1: 2
Enter a string (or 'null' to simulate null pointer exception):
null
NullPointerException: Cannot perform operations on a null reference.

Test Case - 2
User Output
Enter the numerator:
3
Enter the denominator:
0
ArithmeticException: Division by zero is not allowed.
Enter the index to access in the array:
6

```
ArrayIndexOutOfBoundsException: Index is out of bounds.
Enter a string (or 'null' to simulate null pointer exception):
hello
Length of the string: 5
```

S.No: 20	Exp. Name: ***Implementation of Built In Exceptions***	Date: 2026-10-04

Aim:

Write a Java program that reads an integer input from the user and prints the input. If the user enters anything other than an integer, your program should print "invalid input".

Input Format:

- The program will take a single input from the user. This input should be a string which may or may not represent an integer.

Output Format:

- If the input is a valid integer, print the integer.
- If the input is not a valid integer, print "invalid input".

Source Code:

q56419/BuiltinException.java

```
package q56419;
import java.util.Scanner;

class BuiltinException {
        // Write your code here...
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);
                String input = sc.nextLine();
                try{
                        int number = Integer.parseInt(input);
                        System.out.println(number);
                }catch(NumberFormatException e){
                        System.out.println("invalid input");
                }
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5

5

Test Case - 2
User Output
#
invalid input

S.No: 21	Exp. Name: ***User Defined Exceptions***	Date: 2026-10-04

Aim:

Write a Java program that checks if a person is eligible to vote based on their age.

- If the age is below 18 or above 60, throw a custom exception with the message: "Age must be between 18 and 60"
- If the age is 18 or above, but below 60, print "eligible to vote"
- Handle any invalid input gracefully (e.g., non-integer inputs).

You need to create a custom exception class named **VotingEligibilityException** to handle the age validation.

Input Format:

- A single integer age is provided as input.

Output Format:

- If the age is less than 18 or greater than 60, print: **"Age must be between 18 and 60".**
- If the age is valid ($18 \leq age \leq 60$), print: **"eligible to vote".**
- If the input is not an integer, print: **"Invalid input".**

Note: This question does not have partial weightages for test cases. All the test cases must be passed.

Source Code:

q56420/VotingEligibility.java

```
package q56420;
import java.util.Scanner;
public class VotingEligibility {
        // Write your code here...
        static class VotingEligibilityException extends
Exception{
                public VotingEligibilityException(String message)
{
                        super(message);
                }
        }
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);
                try{
                        String input = sc.nextLine();
                        int age = Integer.parseInt(input);

                        if(age < 18 || age > 60){
                                throw new
VotingEligibilityException(
                                        "Age must be between 18
and 60"
                                );
                        }
                        System.out.println("eligible to vote");
                }catch(VotingEligibilityException e){
                        System.out.println(e.getMessage());
                }catch(NumberFormatException e){
                        System.out.println("Invalid input");
                }
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

18
eligible to vote

Test Case - 2
User Output
65
Age must be between 18 and 60

Test Case - 3
User Output
ab6
Invalid input

S.No: 23	Exp. Name: ***Lifecycle of Thread using isAlive() and join()***	Date: 2026-10-04

Aim:

Write a Java program to demonstrate the life cycle of a thread using the methods `isAlive()` and `join()`. The program should perform the following:

1. Create a class named **MyThread** that extends **Thread**.
   - It should contain a private integer attribute **number**.
   - Provide a parameterized constructor that initializes this attribute.
   - Override the **run()** method to simulate some work by calling **Thread.sleep(1000)**.
   - Print the square of the number in the following format:

```
Square of <number> is <square>
```

Input Format:

- A single integer n, the number for which the square needs to be calculated.

Output Format:

Your program must print four things in this exact order:

- Whether the thread is alive before it starts - This will print "true" or "false".
- The square of the number - Printed after the thread finishes its work
- Whether the thread is alive after it has completed - This will again print "true" or "false".
- A final message -"Finished".

Note:

- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

ID: 25ME1A5483

Ramachandra College of Engineering

q57569/ThreadDemo.java

```
package q57569;
import java.util.Scanner;
    // Write your code here...
class MyThread extends Thread{
        private int number;

        MyThread(int number){
                this.number = number;
        }
        public void run(){
                try{
                        Thread.sleep(1000);
                }catch(InterruptedException e){

                }
                System.out.println("Square of " + number + " is "
+ (number * number));
        }
}
public class ThreadDemo {
        // Write your code here...
        public static void main(String[] args)throws Exception{
                Scanner sc = new Scanner(System.in);
                System.out.print("Enter number: ");
                int n = sc.nextInt();
                MyThread t = new MyThread(n);
                System.out.println(t.isAlive());
                t.start();
                t.join();
                System.out.println(t.isAlive());
                System.out.println("Finished");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number:
5

Ramachandra College of Engineering | 2025-2029-AIDS-B | ID: 25ME1A5483 | Page No: 68

false
Square of 5 is 25
false
Finished

Test Case - 2
User Output
Enter number:
7
false
Square of 7 is 49
false
Finished

S.No: 24	Exp. Name: ***Demonstrate the Concept of Daemon Threads***	Date: 2026-10-04

Aim:

Write a Java program to demonstrate the concept of daemon threads. The program should create three threads, where one is explicitly set as a daemon thread, and the others are user threads. Each thread should print whether it is a daemon thread or not.

Input Format:

- No input is required from the user.

Output Format:

- A sequence of lines, each indicating whether a thread is a daemon thread or not.
- For the daemon thread, the output should be: "This is daemon thread".
- For non-daemon (user) threads, the output should be: "This is not a daemon thread".
- The output may appear in any order due to the concurrent execution of threads.

Note:

- Refer to the visible test case to strictly match the input/output layout.

Source Code:

q11351/DaemonThreadDemo.java

```
package q11351;
import java.util.Scanner;
// Write your code here...
class MyThread extends Thread{
        private boolean daemon;

        MyThread(String name,boolean daemon){
                super(name);
                this.daemon = daemon;
        }
        public void run(){
                System.out.println(daemon ? "This is daemon
thread":"This is not a daemon thread");
        }
}
public class DaemonThreadDemo{
        public static void main(String[] args){

                MyThread t1 = new MyThread("Thread-1",true);
                MyThread t2 = new MyThread("Thread-2",false);
                MyThread t3 = new MyThread("Thread-3",false);

                t1.setDaemon(true);

                t1.start();
                t2.start();
                t3.start();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
This is daemon thread
This is not a daemon thread
This is not a daemon thread

Ramachandra College of Engineering | 2025-2029-AIDS-B | ID: 25ME1A5483 | Page No: 71

S.No: 25	Exp. Name: ***Producer and Consumer Problem***	Date: 2026-10-02

Aim:

Write a Java program to implement the Producer-Consumer problem using multithreading and synchronization.

Program Requirements:

1. Prompt the user to enter the number of items n to be produced and consumed.
2. Use separate threads for:
   - Producer: produces items numbered from 0 to $n - 1$
   - Consumer: consumes the produced items in order
3. Implement a shared resource class (**Product**) that:
   - Allows the producer to put an item only when the previous item has been consumed.
   - Allows the consumer to get an item only when a new item has been produced.
4. Print:
   - "PUT: <number>" when the producer places an item
   - "GET: <number>" when the consumer consumes an item
5. Use proper synchronization (`synchronized, wait(), notify()`) to avoid race conditions,to coordinate the producer and consumer without race conditions.

Input Format:

- A single integer n representing how many items should be produced and consumed as:

```
number of items to be produced and consumed: <user_input>
```

Output Format:

The output displays producer and consumer actions:

- Producer: "PUT: <number>" for each item produced (0 to $n - 1$).
- Consumer: "GET: <number>" for each item consumed in order (0 to $n - 1$).

The execution order will alternate correctly due to synchronization.

Constraint:

- $n \geq 0$

Note: Refer to visible test cases for print statements and a better understanding.

Source Code:

q63229/ProdCons.java

```
package q63229;
import java.util.Scanner;

class Product {
        // Write your code here...
        private int item;
        private boolean available = false;
        synchronized void put(int item){
                try{
                        while (available){
                                wait();
                        }
                        this.item = item;
                        System.out.println("PUT: "+item);
                        available = true;
                        notify();
                }
                catch(InterruptedException e){
                        Thread.currentThread().interrupt();
                }
        }
        synchronized int get(){
                try{
                        while(!available){
                                wait();
                        }
                        int value = item;
                        System.out.println("GET: "+value);
                        available = false;
                        notify();
                        return value;
                }
                catch(InterruptedException e){
                        Thread.currentThread().interrupt();
                        return -1;
                }
        }
}
class producer extends Thread{
        private Product product;
        private int n;
        producer(Product product,int n){
                this.product = product;
                this.n = n;
```

```
        }
        public void run(){
                for(int i = 0;i < n;i++){
                        product.put(i);
                }
        }
}
class consumer extends Thread{
        private Product product;
        private int n;
        consumer(Product product,int n){
                this.product = product;
                this.n = n;
        }
        public void run(){
                for(int i = 0;i < n;i++){
                        product.get();
                }
        }
}
public class ProdCons{
        public static void main(String[] args){
                Scanner sc = new Scanner(System.in);
                System.out.print("number of items to be produced
and consumed: ");
                int n = sc.nextInt();
                Product product = new Product();
                producer producer = new producer(product,n);
                consumer consumer = new consumer(product,n);
                producer.start();
                consumer.start();
                try{
                        producer.join();
                        consumer.join();
                }
                catch(InterruptedException e){
                        Thread.currentThread().interrupt();
                }
                sc.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
number of items to be produced and consumed:
5
PUT: 0
GET: 0
PUT: 1
GET: 1
PUT: 2
GET: 2
PUT: 3
GET: 3
PUT: 4
GET: 4

Test Case - 2
User Output
number of items to be produced and consumed:
1
PUT: 0
GET: 0

S.No: 26	Exp. Name: ***Demonstrate the Usage of Package***	Date: 2026-10-02

Aim:

Write a Java program to create a **Calculator** class that performs basic arithmetic operations using methods.

The **Calculator** class should contain the following methods:

- **add()** – accepts two integers and returns their sum.
- **subtract()** – accepts two integers and returns the difference between the first and second integers.

Create a **Main** class containing the **main()** method. The program should read two integers from the user, create an object of the **Calculator** class, and use its methods to perform addition and subtraction.

- Finally, display the results of the two operations.

Input Format:

- The program will take two integers as input, each on a new line.

Output Format:

- The first line of output will display the result of the addition of the two input integers.
- The second line of output will show the result of the subtraction of the two input integers.

Source Code:

q113127/Main.java

```
package q113127;
import java.util.Scanner;

// Write your code here...

public class Main {
    // Write your code here...
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                int num1 = sc.nextInt();
                int num2 = sc.nextInt();
                System.out.println(num1 + num2);
                System.out.println(num1 - num2);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10
20
30
-10

Test Case - 2
User Output
30
5
35
25

Ramachandra College of Engineering 2025-2029-AIDS-B ID: 25ME1A5483 Page No: 77

S.No: 27	Exp. Name: ***Maximum and Minimum Values in an Array***	Date: 2026-09-29

Aim:

Write a Java program to find both the maximum and minimum elements in an array of n integers.

Input Format:

- The first line contains an integer n, representing the size of the array.
- The second line contains n space-separated integers representing the array elements.

Output Format:

- The first line must contain the maximum element found in the array.
- The second line must contain the minimum element found in the array.

Source Code:

q82113/MaxMinArray.java

```
package q82113;
import java.util.Scanner;

public class MaxMinArray {

	public static void main(String args[]){
		Scanner sc = new Scanner(System.in);
		int size = sc.nextInt();
		int[] arr = new int[size];
		for(int i = 0;i < size;i++){
			arr[i] = sc.nextInt();
		}
		int max = arr[0];
		int min = arr[0];
		for(int i = 0;i < size;i++){
			if(max < arr[i]){
				max = arr[i];
			}
			if(min > arr[i]){
				min = arr[i];
			}
		}
		System.out.println(max);
		System.out.println(min);
	}
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
10 25 7 40 18
40
7

Test Case - 2
User Output

6
1 2 3 4 5 6
6
1

Test Case - 3
User Output
6
15 -21 45 32 -20 36
45
-21

S.No: 28	Exp. Name: ***Even and Odd Numbers up to a Given Range***	**Date: 2026-09-29**

Aim:

Write a Java program to print all even and odd numbers from 1 to a given range.

Input Format:

- The input consists of a single integer n, which represents the upper limit of the range.

Output Format:

- The first line should print all even numbers from 1 to n, separated by spaces.
- The second line should print all odd numbers from 1 to n, separated by spaces.

Note: Print a space after every number, including the last number in each line, followed by a newline.

Source Code:

q110112/EvenOddPrinter.java

```
package q110112;
import java.util.Scanner;

public class EvenOddPrinter {
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                int num = sc.nextInt();
                for(int i = 1;i <= num;i++){
                        if(i % 2 == 0){
                                System.out.print(i);
                                if(i <= num){
                                        System.out.print(" ");
                                }
                        }
                }
                System.out.println();
                for(int i = 1;i <= num; i++){
                        if(i % 2 != 0){
                                System.out.print(i);
                                if(i <= num){
                                        System.out.print(" ");
                                }
                        }
                }
                System.out.println();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
2 4
1 3 5

Test Case - 2
User Output

10
2 4 6 8 10
1 3 5 7 9

S.No: 29	Exp. Name: ***Skip by 2 Sequence***	Date: 2026-10-02

Aim:

Write a Java program to print a sequence of numbers from a given start value to an end value (both inclusive), incrementing by 2 at each step.

Input Format:

- A single line containing two space-separated integers: start and end.

Output Format:

- A single line containing the space-separated integers of the generated sequence.

Note: Do not print any trailing space after the last element. The output should end with a newline.

Source Code:

q110111/SkipByTwo.java

```
package q110111;
import java.util.Scanner;

public class SkipByTwo {
        // Write your code here...
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                int start = sc.nextInt();
                int end = sc.nextInt();
                while(start <= end){
                        System.out.print(start);
                        start = start + 2;
                        if(start <= end){
                                System.out.print(" ");
                        }
                }
                System.out.println();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 10
1 3 5 7 9

Test Case - 2
User Output
5 5
5

S.No: 30	Exp. Name: ***Reverse a Number***	Date: 2026-10-02

Aim:

Write a Java program to reverse a given positive integer n using modular arithmetic.

Input Format:

- A single line containing a positive integer n.

Output Format:

- Print the reversed number.

Note:

- Zeros in the original number should be dropped in the reversed number.

Source Code:

q93537/ReverseNumber.java

```
package q93537;
import java.util.Scanner;

public class ReverseNumber {
		// Write your code here....
	public static void main(String args[]){
		Scanner sc = new Scanner(System.in);
		int num = sc.nextInt();
		int rev = 0;
		while(num != 0){
			rev = rev * 10 + num % 10;
			num /= 10;
		}
		System.out.println(rev);
	}
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
456
654

ID: 25ME1A5483
2025-2029-AIDS-B
Ramachandra College of Engineering

Test Case - 2
User Output
1234
4321

Test Case - 3
User Output
100
1